

Báo cáo Nghiên cứu Tối ưu hóa vLLM Đột phá trên Hạ tầng Hạn chế: NVIDIA H200 MIG 18GB VRAM và 3 CPU Cores

Giới thiệu và Phân tích Căn nguyên Nút thắt Cổ chai (Bottleneck Analysis)

Việc phục vụ (serving) Mô hình Ngôn ngữ Lớn (LLM) trong môi trường sản xuất thông qua nền tảng vLLM thường được thiết kế với giả định rằng hệ thống sở hữu tài nguyên phần cứng dồi dào, đặc biệt là sự tương xứng giữa năng lực tính toán của GPU và số lượng lõi xử lý của CPU. Tuy nhiên, bài toán đặt ra yêu cầu triển khai trên một môi trường cực đoan và phi tiêu chuẩn: một lát cắt MIG (Multi-Instance GPU) của kiến trúc NVIDIA H200 với dung lượng VRAM bị giới hạn ở mức 18GB, đi kèm với tài nguyên tính toán từ hệ điều hành chỉ có vỏn vẹn 3 CPU cores. Đây là một cấu hình tạo ra "nút thắt kép" (double bottleneck) hiếm gặp trong kỹ thuật AI Infrastructure, nơi mà các chiến lược tối ưu hóa truyền thống thường phản tác dụng.

Mặc dù đội ngũ kỹ sư đã áp dụng các biện pháp tiêu chuẩn như lượng tử hóa FP8 cho cả trọng số mô hình và bộ nhớ đệm KV (KV cache), kích hoạt tính năng chunked prefill, sử dụng prefix caching và tinh chỉnh các thông số cơ bản, hệ thống vẫn chạm ngưỡng giới hạn vật lý của hai chỉ số quan trọng nhất: TTFT (Time to First Token - Độ trễ đến token đầu tiên) và TPOT (Time Per Output Token - Độ trễ sinh từng token). Sự trì trệ này không xuất phát từ việc GPU H200 thiếu năng lực toán học. Kiến trúc Hopper (SM90) của H200 sở hữu bộ nhớ băng thông cực cao và Tensor Cores thế hệ thứ tư vượt trội¹. Vấn đề cốt lõi nằm ở hiện tượng "đói lệnh" (kernel launch starvation).

Hệ thống vLLM phụ thuộc rất lớn vào CPU cho hàng loạt các tác vụ song song: định tuyến API (API routing), mã hóa và giải mã token (tokenization/detokenization), quản lý bộ nhớ PagedAttention, và quan trọng nhất là vòng lặp lập lịch (scheduler loop)³. Với chỉ 3 CPU cores, tài nguyên bị cạn kiệt do chi phí chuyển đổi ngữ cảnh (context switching) của hệ điều hành và sự tắc nghẽn liên tục tại khóa GIL (Global Interpreter Lock) trong Python⁵. Sự chậm trễ ở CPU dẫn đến việc GPU phải nằm chờ các chỉ thị thực thi tiếp theo, trực tiếp làm tăng TPOT¹. Đồng thời, giới hạn 18GB VRAM tạo ra áp lực khổng lồ lên hệ thống phân bổ, buộc bộ lập lịch (scheduler) phải thực hiện các tác vụ dọn dẹp, hoán đổi (swapping) và tính toán lại (recomputation) liên tục, càng làm trầm trọng thêm tình trạng quá tải của CPU⁸. Báo cáo này sẽ trình bày các kỹ thuật tối ưu hóa phi truyền thống, can thiệp sâu vào vi kiến trúc của vLLM nhằm giải phóng CPU, tái cấu trúc VRAM và thiết lập các giới hạn vật lý mới để tối đa hóa hiệu năng phục vụ.

Tái cấu trúc Kiến trúc Đa tiến trình và Triệt tiêu Overhead Hệ điều hành

Khung ứng dụng vLLM được thiết kế mặc định với mô hình đa tiến trình (multiprocessing) nhằm phân tách lớp tiếp nhận yêu cầu (OpenAI API server frontend) và lớp suy luận lõi (Model Engine backend)¹¹. Mặc dù mô hình này giúp tăng cường khả năng mở rộng trên các máy chủ khổng lồ, nó lại trở thành một điểm nghẽn chí mạng trong môi trường bị giới hạn nghiêm ngặt ở 3 CPU cores.

Vô hiệu hóa Frontend Multiprocessing

Theo kiến trúc mặc định, Uvicorn/FastAPI (đóng vai trò là API Server) chạy trên một tiến trình hệ điều hành riêng biệt và giao tiếp với AsyncLLMEngine thông qua cơ chế giao tiếp liên tiến trình (IPC - Inter-Process Communication), thường dựa trên ZeroMQ (ZMQ) hoặc các đường ống bộ nhớ chia sẻ¹². Việc truyền tải hàng ngàn yêu cầu văn bản giữa tiến trình API và tiến trình Engine đòi hỏi CPU phải liên tục thực hiện các thao tác sao chép bộ nhớ (memory copying), giải mã và mã hóa JSON (JSON deserialization/serialization), cũng như chịu đựng độ trễ của việc chuyển đổi ngữ cảnh tiến trình (process context switching)¹⁵. Trên một hệ thống chỉ có 3 cores, các tiến trình này sẽ tranh chấp thời gian thực thi một cách bạo lực, dẫn đến hiện tượng "thrashing" — nơi CPU tốn nhiều thời gian để chuyển đổi giữa các tác vụ hơn là thực sự xử lý chúng. Giải pháp đột phá đầu tiên là cấu hình lại vòng đời của ứng dụng bằng cách khởi chạy vLLM với cờ --disable-frontend-multiprocessing¹⁶. Khi tham số này được kích hoạt, máy chủ frontend OpenAI sẽ được nhúng trực tiếp và chạy trong cùng một không gian tiến trình với engine phục vụ mô hình¹³. Hệ quả của kỹ thuật này đối với môi trường 3 cores là sự thay đổi hoàn toàn về đặc tính hiệu năng. Độ trễ IPC được triệt tiêu hoàn toàn, cho phép dữ liệu yêu cầu từ mạng (network request) đi trực tiếp vào vòng lặp sự kiện (event loop) của AsyncIO mà không cần phải chờ đợi thông qua các hàng đợi tiến trình trung gian⁶. Hơn thế nữa, bằng cách giữ mọi luồng dữ liệu trong cùng một không gian địa chỉ ảo, bộ nhớ đệm L1/L2 của CPU không bị xóa bỏ (flush) liên tục, giúp các thao tác tính toán metadata diễn ra với tốc độ tối đa.

Thông số Kiến trúc	Chế độ Mặc định (Multiprocessing)	Chế độ Tối ưu (Uni-process)	Tác động lên Môi trường 3 Cores
Giao tiếp Frontend-Engine	IPC (ZeroMQ / Shared Memory)	Hàm gọi trực tiếp (Direct Async Call)	Triệt tiêu overhead sao chép dữ liệu và JSON parsing.
Chuyển đổi Ngữ cảnh OS	Cao (Tranh chấp giữa 2+ tiến trình)	Thấp (Chỉ quản lý luồng trong 1 tiến trình)	Bảo toàn bộ nhớ đệm L1/L2, giảm Kernel space CPU time.
Sử dụng CPU Cores	Phân mảnh, gây thất cổ chai	Tập trung, kiểm soát tuyến tính	Dành 1 core cho I/O, 1 core cho Scheduler, 1 core

			cho OS.
--	--	--	---------

Loại bỏ Distributed Executor Backend và Cấu hình Đơn Tiến Trình

Ngay cả khi hệ thống chỉ nhận diện 1 GPU thông qua cấu hình MIG, cơ chế nội bộ của vLLM đôi khi vẫn cố gắng khởi tạo các thành phần giao tiếp phân tán (distributed backend) như Ray hoặc các tiến trình nền của torch.distributed tùy thuộc vào môi trường thư viện¹⁹. Kiến trúc Ray, dù mạnh mẽ cho việc điều phối cụm (cluster orchestration), lại liên tục tiêu tốn chu kỳ CPU cho các tác vụ nền như kiểm tra trạng thái sức khỏe (health checks), thu thập rác (garbage collection) và duy trì kết nối heartbeat²⁰. Đối với một CPU chỉ có 3 cores, việc mất đi một phần nhỏ tài nguyên cho các tác vụ quản lý cụm ảo này là không thể chấp nhận được. Việc ép buộc vLLM sử dụng trình thực thi cục bộ thông qua tham số `--distributed-executor-backend mp` hoặc cấu trúc hệ thống để sử dụng UniProcExecutor (Single-process model execution) là bắt buộc²¹. Kiến trúc UniProcExecutor đảm bảo rằng vòng lặp lập lịch và việc đẩy các kernel CUDA xuống GPU diễn ra tuần tự, mạch lạc trên cùng một luồng duy nhất, loại bỏ hoàn toàn các cấu trúc RPC (Remote Procedure Call)⁶.

Chế ngự Sự Quá tải Luồng của OpenMP và MKL

Bên dưới lớp vỏ của vLLM, thư viện PyTorch sử dụng các bộ gia tốc toán học trên CPU như OpenMP (thông qua libgomp hoặc libiomp5) và Intel MKL để thực hiện các phép toán tensor chưa được offload xuống GPU¹⁵. Đặc tính nguy hiểm nhất của các thư viện này là chúng thường tự động khởi tạo một pool luồng (thread pool) có số lượng bằng với tổng số lõi logic của toàn bộ máy chủ vật lý, bỏ qua hoàn toàn các giới hạn ảo hóa hay cgroup mà hệ thống cấp phát (trong trường hợp này là 3 cores)²³. Hậu quả là hàng chục luồng ảo được sinh ra và cố gắng chen lấn để thực thi trên 3 lõi vật lý. Tình trạng oversubscription (đăng ký quá mức) này làm suy sụp hoàn toàn hiệu năng xử lý³. Để ngăn chặn điều này, trước khi khởi chạy tiến trình vLLM, bắt buộc phải áp đặt các ranh giới thực thi mức thấp nhất của hệ điều hành Linux thông qua các biến môi trường: `OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, và `OPENBLAS_NUM_THREADS=1`²³. Bằng cách chuyển đổi các tác vụ phụ trợ này về dạng tính toán đơn luồng (single-threaded math), hệ thống bảo toàn được sức mạnh của 2 cores còn lại dành riêng cho vòng lặp Event Loop của vLLM và quá trình giao tiếp PCIe với GPU.

Chiến lược Phân bổ VRAM và Nghịch lý CUDA Graphs

Giải quyết nút thắt CPU dẫn chúng ta đến một rào cản vật lý khác: 18GB VRAM của phân vùng MIG H200. Một trong những thách thức lớn nhất của quá trình giải mã (decoding) trong suy luận LLM là chi phí khởi chạy kernel (kernel launch overhead)¹. Với CPU yếu, nếu PyTorch hoạt động ở chế độ Eager mode, CPU sẽ phải liên tục biên dịch và gửi hàng ngàn lệnh rời rạc tới GPU cho mỗi token được sinh ra. CPU sẽ nhanh chóng tụt lại phía sau, khiến GPU H200 mạnh mẽ rơi vào trạng thái chờ (GPU idle), làm TPOT tăng vọt¹⁵. Để giải phóng CPU khỏi tác vụ này, vLLM sử dụng cơ chế **CUDA Graphs**. Kỹ thuật này "chụp"

(capture) toàn bộ biểu đồ thực thi của một chuỗi các kernel CUDA trong giai đoạn khởi tạo, sau đó gói chúng thành một lệnh gọi duy nhất từ CPU trong thời gian chạy⁶. Tính năng này giảm tải cho CPU tới 90% trong giai đoạn decode.

Tuy nhiên, CUDA Graphs đòi hỏi phải bảo lưu một lượng VRAM tĩnh khổng lồ trên GPU²⁷. Trong không gian 18GB VRAM (nơi mô hình FP8 chiếm khoảng 8GB và overhead cơ bản chiếm khoảng 1.5GB), không gian còn lại cho KV Cache chỉ vào khoảng 8.5GB. Mặc định, vLLM sẽ xây dựng các đồ thị CUDA cho hàng loạt các kích thước batch khác nhau (ví dụ: 1, 2, 4, 8, 16, 32, 64...), mỗi đồ thị lại chiếm một phần VRAM tĩnh²⁷. Việc này nhanh chóng dẫn đến lỗi Out-Of-Memory (OOM).

Tối ưu Hóa Ngược: Bóp Nghẹt Không Gian Yêu Cầu Để Bảo Vệ CUDA Graphs

Để giải quyết mâu thuẫn giữa việc cần CUDA Graphs để cứu CPU và thiếu VRAM để chứa chúng, một chiến lược "tối ưu hóa ngược" phải được áp dụng. Thay vì tối đa hóa số lượng chuỗi đồng thời (max_num_seqs) để tăng thông lượng như các tài liệu thông thường khuyến nghị, chúng ta phải chủ động bóp nghẹt nó.

Bằng cách thiết lập --max-num-seqs 16 hoặc --max-num-seqs 32, chúng ta buộc vLLM giới hạn số lượng yêu cầu xử lý đồng thời trong một bước của bộ lập lịch²⁹. Khi giới hạn này được hạ thấp, vLLM sẽ dừng việc biên dịch và lưu trữ các biến thể CUDA Graphs cho các batch lớn (như 64, 128 hay 256), tiết kiệm ngay lập tức hàng Gigabyte VRAM tĩnh²⁷. Đồng thời, ta đẩy tham số --gpu-memory-utilization lên mức cực đoan (0.95 đến 0.98), vì việc chạy trên lát cắt MIG độc lập đảm bảo không có tiến trình ngoại vi nào tranh giành VRAM²⁸.

Việc giảm max_num_seqs tuy hạn chế thông lượng tổng thể của hệ thống, nhưng lại bảo vệ thành công tính năng CUDA Graphs. CPU không bị quá tải bởi việc gửi lệnh (kernel dispatch), VRAM không bị tràn (OOM), và độ trễ sinh từng token (TPOT) được duy trì ở mức cực thấp, đáp ứng chính xác bài toán phục vụ ưu tiên độ trễ.

Tham số Tối ưu Bộ nhớ	Giá trị Khuyến nghị	Cơ chế Hoạt động và Tác động Vật lý
gpu_memory_utilization	0.96	Tận dụng tối đa VRAM vật lý 18GB của MIG slice, chừa lại 4% (khoảng 720MB) cho các cấu trúc dữ liệu PyTorch động.
max_num_seqs	16	Giảm số lượng đồ thị CUDA Graph phải lưu trữ, giải phóng hàng Gigabyte VRAM tĩnh để nhường chỗ cho PagedAttention KV

		Cache.
enforce_eager	False	Bắt buộc bật CUDA Graphs. Bỏ qua cờ này sẽ giải phóng thêm 1GB VRAM nhưng phá hủy hiệu năng của CPU 3 cores do overhead gửi lệnh.

Tối ưu Hóa Attention Backend và Giải phóng CPU qua GPU Offloading

Khi CPU là một hạn chế vật lý cứng, triết lý thiết kế hệ thống phải chuyển hướng sang việc đẩy mọi tác vụ tính toán không mang tính chất điều phối (non-orchestration computation) xuống GPU, bất chấp việc GPU có thể chưa được sử dụng hết công suất lý thuyết.

Kiến trúc Hopper và Sự Thống trị của FlashInfer

Kiến trúc Hopper (sm_90a) của NVIDIA H200 là một bước nhảy vọt về vi kiến trúc phần cứng, được trang bị Tensor Memory Accelerator (TMA) cho phép truyền tải dữ liệu đa chiều bất đồng bộ giữa bộ nhớ HBM (High Bandwidth Memory) và SRAM (Shared Memory) trên chip mà không cần luồng CUDA phải can thiệp³². Mặc dù FLASH_ATTN thường là backend mặc định được vLLM tự động lựa chọn, nhưng trong các kịch bản giải mã (decode) liên tục với batch size nhỏ và KV Cache được lượng tử hóa FP8, thư viện **FlashInfer** thể hiện sự vượt trội rõ rệt¹.

FlashInfer được thiết kế đặc biệt để tối ưu hóa Paged KV Cache và Grouped Query Attention (GQA)³³. Bằng cách sử dụng FlashInfer thông qua biến môi trường

VLLM_ATTENTION_BACKEND=FLASHINFER (và củng cố bằng

VLLM_FLASH_ATTN_VERSION=3), hệ thống tận dụng tối đa TMA của H200³⁶. FlashInfer có khả năng thực thi PagedAttention mà không cần bộ xử lý trung tâm (CPU) phải liên tục cấu trúc lại các con trỏ bộ nhớ trung gian, giúp tiết kiệm băng thông PCIe và giảm tải quản lý từ phía CPU³³.

Kết quả là tốc độ giải mã (decode speed) có thể tăng trưởng từ 1.5x đến 2.2x so với

FlashAttention thông thường trên cùng phần cứng Hopper¹.

Chuyển giao Hoàn toàn Bộ lấy Mẫu (FlashInfer Sampler)

Một trong những quy trình gây tổn kém CPU nhất nhưng ít được chú ý là giai đoạn lấy mẫu (sampling). Theo cơ chế truyền thống, sau khi GPU tính toán xong mạng nơ-ron và tạo ra vector logit của token tiếp theo, vector này (thường có kích thước bằng với toàn bộ từ vựng của LLM) phải được sao chép ngược qua bus PCIe về bộ nhớ của CPU⁶. CPU sau đó sử dụng thư viện PyTorch để thực hiện các phép toán phức tạp như Softmax, Nucleus Sampling (Top-P), Top-K, và áp dụng các hình phạt (penalties), sau đó sinh số ngẫu nhiên (RNG) để chọn ra token cuối cùng. Mã ID của token này sau đó mới được gửi ngược lại GPU cho vòng lặp tiếp theo⁶.

Quá trình "ping-pong" dữ liệu giữa Host (CPU) và Device (GPU), kết hợp với việc 3 CPU cores

phải xử lý toán học mảng lớn, tạo ra một độ trễ chỉ mạng cản trở TPOT¹⁵. Rất may, vLLM gần đây đã tích hợp khả năng offload toàn bộ quy trình này xuống GPU. Bằng cách thiết lập cờ `VLLM_USE_FLASHINFER_SAMPLER=1`, toàn bộ quá trình Top-K, Top-P, Softmax và sinh số ngẫu nhiên đều được thực thi song song bởi các luồng CUDA chuyên dụng trên H200³⁶. CPU được giải phóng hoàn toàn khỏi các bộ xử lý Logits (Logits processors), qua đó loại bỏ sự phụ thuộc vào cấu hình đa luồng `VLLM_LOGITS_PROCESSOR_THREADS` (có thể ép về 0 một cách an toàn)³⁶. Kỹ thuật này trực tiếp cắt giảm hàng chục mili-giây độ trễ từ mỗi vòng lặp sinh token.

Đột phá Lịch trình: Tinh chỉnh PagedAttention và Nghịch lý Chunked Prefill

Trong kỹ thuật tối ưu hóa LLM, việc tinh chỉnh hệ thống quản lý bộ nhớ PagedAttention và bộ lập lịch (scheduler) là chìa khóa để cân bằng giữa thông lượng và độ trễ.

Thao tác Kích thước Khối (Block Size Manipulation)

Cơ chế PagedAttention chia KV Cache thành các khối (blocks) vật lý có kích thước cố định, mô phỏng cách hệ điều hành quản lý bộ nhớ ảo. Mặc định trong vLLM, tham số `--block-size` được đặt là 16 token⁹. Với block size nhỏ, sự phân mảnh nội vi (internal fragmentation) — tức là không gian VRAM bị lãng phí ở khối cuối cùng của một chuỗi chưa được lấp đầy — được giữ ở mức tối thiểu⁸.

Tuy nhiên, kích thước khối nhỏ đồng nghĩa với việc số lượng khối (và các con trỏ tương ứng) tăng lên theo cấp số nhân đối với các ngữ cảnh (context) dài. Một prompt có độ dài 4096 tokens với block size 16 sẽ sinh ra một bảng ánh xạ (Block Table) chứa 256 con trỏ⁸. Bộ lập lịch chạy trên CPU phải liên tục duyệt qua hàng ngàn con trỏ này cho mọi yêu cầu đang hoạt động trong mỗi vòng lặp để kiểm tra tính hợp lệ, cấp phát, hoặc thu hồi bộ nhớ⁸. Việc lặp qua các cấu trúc dữ liệu khổng lồ này (pointer chasing) tiêu tốn một lượng lớn chu kỳ đồng hồ của 3 CPU cores, làm chậm quá trình lập lịch.

Kỹ thuật tối ưu: Tăng mạnh `--block-size` lên 32 hoặc 64⁸. Việc tăng kích thước khối gây ra tổn thất nhỏ về VRAM do phân mảnh nội vi tăng lên⁸. Tuy nhiên, trên 18GB VRAM, mất mát khoảng vài chục Megabytes là một sự đánh đổi cực kỳ rẻ để đổi lấy việc kích thước Block Table giảm đi từ 2 đến 4 lần. Bộ lập lịch trên CPU hoàn thành vòng lặp nhanh hơn đáng kể, trực tiếp làm giảm độ trễ giữa các token.

Nghịch lý của Chunked Prefill trên CPU Yếu

Tính năng Chunked Prefill, được bật mặc định trong vLLM V1, là một cơ chế tinh vi nhằm giải quyết vấn đề Head-of-line blocking¹⁰. Khi một yêu cầu với prompt khổng lồ xuất hiện, thay vì buộc hệ thống phải xử lý toàn bộ prompt đó trong một lần (làm đình trệ quá trình sinh token của các yêu cầu khác), vLLM chia prompt đó thành các mảnh (chunks) nhỏ dựa trên giới hạn của `max_num_batched_tokens` và xen kẽ việc xử lý chúng với các bước giải mã của các yêu cầu đang chạy¹⁰.

Tuy nhiên, đối với một CPU chỉ có 3 lõi, cơ chế này lại bộc lộ một điểm yếu chí mạng. Nếu `max_num_batched_tokens` được đặt ở các mức mặc định thấp (như 512 hoặc 2048)¹⁰, một

prompt dài sẽ bị băm vụn thành hàng chục mảnh nhỏ. Việc này đồng nghĩa với việc CPU scheduler phải liên tục ngắt quãng (interrupt), tính toán lại trạng thái, cấu trúc lại đồ thị, và gửi các lệnh CUDA siêu nhỏ xuống GPU lặp đi lặp lại¹⁰. Chi phí quản lý (overhead) của quá trình chia nhỏ này tích lũy và làm tê liệt CPU.

Chiến lược can thiệp: Để bảo vệ CPU, chúng ta phải buộc GPU nuốt các khối dữ liệu lớn hơn trong một lần gửi lệnh, bất chấp việc điều này có thể tạm thời làm chậm các chuỗi khác trong vài mili-giây. Cần tăng mạnh tham số `--max-num-batched-tokens` lên mức 4096 hoặc 8192³¹. Bằng cách mở rộng ngân sách token cho mỗi vòng lặp lập lịch, số lượng các mảnh chunk bị cắt giảm đi đáng kể¹⁰. GPU nhận được một khối lượng công việc đủ lớn để bận rộn trong một khoảng thời gian dài hơn, cho phép CPU "thở" và xử lý các tác vụ mạng hoặc giải phóng bộ nhớ ở chế độ nền thay vì liên tục bị gọi lại bởi bộ lập lịch.

Điểm mù của Prefix Caching

Kỹ thuật Prefix Caching thường được ca ngợi vì khả năng tái sử dụng KV Cache cho các đoạn tiền tố văn bản giống nhau (như system prompts), giúp tăng tốc độ phản hồi. Tuy nhiên, thuật toán đứng sau Prefix Caching yêu cầu CPU phải phân rã chuỗi token đầu vào, tính toán hàm băm mã hóa (thường là SHA-256) trên các đoạn này, và sau đó tra cứu trong bảng băm toàn cục⁴⁴.

Việc tính toán mã băm SHA-256 cho hàng ngàn token là một tác vụ nặng về tính toán số nguyên (integer compute-bound)⁴⁴. Trên cấu hình 3 cores, quá trình này cướp đi hàng chục mili-giây thời gian xử lý, trực tiếp làm tăng TTFT một cách vô hình. Nếu dữ liệu truy vấn đầu vào của hệ thống có tính chất đa dạng cao và ít có đoạn tiền tố dùng chung, việc bật Prefix Caching chỉ mang lại chi phí tính toán mà không thu được lợi ích lưu trữ. Cấu hình tắt hoàn toàn tính năng này (`--no-enable-prefix-caching`) sẽ loại bỏ bước kiểm tra băm trên CPU, giúp yêu cầu đi thẳng vào giai đoạn prefill một cách nhanh nhất⁴⁵.

Giải mã Đầu cơ Không Mô Hình và Suy Thoái Lấy Mẫu (Model-Free Speculation)

Speculative Decoding (Giải mã Đầu cơ) là một trong những cơ chế tăng tốc sinh token mạnh mẽ nhất hiện nay. Về lý thuyết, nó sử dụng một mô hình nhỏ (Draft Model) để dự đoán trước một chuỗi các token, sau đó dùng mô hình lớn (Target Model) để xác thực toàn bộ chuỗi đó trong một lần forward pass duy nhất, biến quá trình sinh văn bản tuần tự thành quá trình xác minh song song, giúp tăng tốc độ sinh token lên gấp 2-3 lần².

Tuy nhiên, trong giới hạn VRAM 18GB ngặt nghèo, việc tải thêm một Draft Model (dù là các biến thể nhỏ nhất cỡ 1B parameter) sẽ chiếm dụng thêm từ 1.5GB đến 2GB VRAM². Sự chiếm dụng này sẽ đẩy KV Cache vào tình trạng cạn kiệt, gây ra các vòng lặp thu hồi (eviction) liên tục hoặc OOM, làm triệt tiêu hoàn toàn lợi ích về tốc độ⁴⁷. Việc phải xử lý thêm các tensor từ mô hình dự đoán cũng tạo thêm gánh nặng cho 3 CPU cores trong khâu điều phối⁴⁷.

Prompt Lookup Decoding (N-Gram Speculation)

Giải pháp duy nhất để khai thác Speculative Decoding mà không tiêu tốn VRAM là sử dụng cơ chế **Model-Free Speculation** (Giải mã Đầu cơ Không Cần Mô Hình), cụ thể là phương pháp tra

cứu Prompt (Prompt Lookup hoặc N-Gram matching)⁴⁷. Phương pháp này không sử dụng mạng nơ-ron phụ, mà dựa vào một thuật toán trích xuất các chuỗi token (N-grams) từ chính đoạn prompt đầu vào hoặc lịch sử văn bản đã sinh ra⁴⁷. Nó giả định rằng văn bản thường chứa các mô hình lặp lại, đặc biệt trong các tác vụ liên quan đến lập trình (code generation), xử lý JSON, hoặc tóm tắt tài liệu (RAG)⁴⁷. Khi tìm thấy sự trùng khớp, hệ thống dự đoán các token tiếp theo dựa trên dữ liệu quá khứ và gửi chúng cho GPU xác thực.

Cấu hình rủi ro cao: Thuật toán tìm kiếm chuỗi lặp lại (linear scan) của Prompt Lookup chạy hoàn toàn trên CPU⁴⁷. Nếu chúng ta cấu hình nó nhìn lại quá sâu vào một ngữ cảnh dài (ví dụ: quét qua 8000 tokens), 3 CPU cores sẽ bị "đóng băng" vì phải thực hiện vô số phép so sánh chuỗi.

Để áp dụng thành công kỹ thuật này, bắt buộc phải giới hạn ranh giới tìm kiếm ở mức tối thiểu:

- Kích hoạt cấu hình: `--speculative-config '{"method": "ngram", "num_speculative_tokens": 3, "prompt_lookup_max": 2}'`⁴⁹.
- Tham số `prompt_lookup_max`: 2 (giảm từ mặc định là 4) đảm bảo CPU chỉ tìm kiếm các mẫu độ dài rất ngắn (2 token)⁴⁹. Việc này tiêu tốn thời gian tính toán gần như bằng không nhưng vẫn cho phép dự đoán cực kỳ chuẩn xác các chuỗi định dạng (như `\n\n`, `{\n`, `def`), mang lại sự cải thiện rõ rệt cho TPOT mà không gây áp lực ngược lên CPU.

Hợp thức hóa việc Đánh đổi Độ chính xác (Acceptable Accuracy Degradation)

Yêu cầu bài toán mở ra khả năng "chấp nhận hy sinh accuracy nhằm tối đa hóa hiệu năng mà không vi phạm luật chống gian lận". Luật chống gian lận trong benchmark hệ thống LLM (Anti-Cheating) nghiêm cấm việc sử dụng bộ nhớ đệm kết quả cứng (static response caching) hoặc trả về các kết quả vô nghĩa từ các mô hình giả mạo. Tuy nhiên, việc tinh chỉnh các **tham số lấy mẫu (Sampling Parameters)** để loại bỏ các phép toán phức tạp là hoàn toàn hợp lệ và phản ánh đúng cách các hệ thống thời gian thực phản ứng dưới tải cao.

1. **Lấy mẫu Tham lam (Greedy Search):** Bằng cách buộc các yêu cầu gửi đến API có `temperature=0.0` (hoặc thiết lập cấu hình bỏ qua `top-k`, `top-p`), engine sẽ mặc định chuyển sang chế độ lấy mẫu tham lam⁵⁰. Lấy mẫu tham lam chỉ là một phép tìm giá trị lớn nhất (argmax) của tensor logit, loại bỏ hoàn toàn các hàm toán học tốn kém (như tính toán xác suất tích lũy) và hàm tạo số ngẫu nhiên (RNG)⁴⁷. Điều này khiến câu trả lời của mô hình trở nên "tắt định" (deterministic) và có thể giảm tính sáng tạo, nhưng đổi lại, thời gian thực thi của bước hậu xử lý (post-processing) giảm thiểu đáng kể⁵¹. Nó cũng giúp tối đa hóa tỷ lệ chấp nhận (acceptance rate) của thuật toán N-Gram Speculative Decoding vì loại trừ được sự phân kỳ phân phối xác suất do tính ngẫu nhiên⁴⁷.
2. **Loại bỏ Tính toán Logprobs:** Phải đảm bảo tham số `logprobs` không bao giờ được cấu hình hoặc yêu cầu (`--max-logprobs` được tắt)¹³. Nếu `logprobs` được yêu cầu, vLLM phải lấy toàn bộ tensor xác suất kích thước lớn từ GPU, gửi qua băng thông PCIe về CPU, sắp xếp, lọc, và cuối cùng mã hóa chúng thành một cấu trúc JSON phức tạp để trả về mạng¹⁴. Việc vô hiệu hóa hoàn toàn truy xuất `logprobs` loại bỏ gánh nặng tuần tự hóa (serialization) này, giải cứu CPU I/O và tăng tốc chu kỳ trả về API.

Tối đa hóa Tiềm năng I/O bằng cách Đóng băng Viễn thám (Observability Freeze)

Trong môi trường điện toán mây hoặc các cụm máy chủ lớn, việc thu thập số liệu phân tích và viễn thám (telemetry) là không thể thiếu. Tuy nhiên, đối với một cấu hình bị bóp nghẹt ở 3 CPU cores, các tác vụ giám sát nền này là sự lãng phí tài nguyên xa xỉ. Hệ thống ghi nhật ký (logging) và điểm cuối Prometheus trong vLLM sử dụng các luồng không đồng bộ liên tục truy xuất trạng thái của GPU và bảng PagedAttention để tổng hợp báo cáo trạng thái hệ thống¹⁰. Việc định dạng các chuỗi báo cáo này (ví dụ: Avg prompt throughput: X tokens/s) và xuất ra I/O yêu cầu các System Calls từ nhân hệ điều hành, gây ra các khoảng dừng (blocking) ngắn nhưng chí mạng, cản trở việc thực thi của luồng chính.

Để đạt được hiệu suất cực đoan, hệ thống cần được chuyển sang trạng thái "đóng băng khả năng quan sát" (Observability Freeze):

- Tích hợp cờ `--disable-log-stats` để dừng hoàn toàn việc in nhật ký thống kê trạng thái định kỳ mỗi 5 giây, đồng thời dừng cập nhật các chỉ số nội bộ của Prometheus¹⁶.
- Tích hợp cờ `--disable-log-requests` để ngăn chặn việc ghi nhật ký Access Log đối với mỗi HTTP request nhận được và hoàn thành, giảm tải cho bộ đệm I/O của đĩa và CPU¹⁶.
- Thiết lập các biến môi trường mức hệ thống để ngăn cản mọi tiến trình nền báo cáo việc sử dụng về máy chủ trung tâm: `VLLM_NO_USAGE_STATS=1` và `VLLM_DO_NOT_TRACK=1`³⁷.

Kết luận và Cấu hình Khuyến nghị Triển khai Thực tế (Actionable Deployment Configuration)

Bài toán phục vụ mô hình ngôn ngữ lớn trên hạ tầng hạn chế tài nguyên phi đối xứng (GPU Hopper hiện đại kết hợp với 18GB VRAM nhỏ hẹp và 3 CPU cores yếu ớt) không thể được giải quyết bằng các quy tắc tối ưu hóa thông thường. Bằng cách can thiệp sâu vào kiến trúc tiến trình của vLLM, viết lại các quy tắc cấp phát bộ nhớ, đẩy toàn bộ gánh nặng toán học xuống GPU, tinh chỉnh các nghịch lý của bộ lập lịch, và mạnh tay loại bỏ các luồng viễn thám, hệ thống có thể được giải phóng khỏi các điểm nghẽn nội tại.

Để hiện thực hóa toàn bộ các phân tích đột phá trên, dưới đây là bộ tham số thực thi cấu hình chi tiết, nhằm tối đa hóa hiệu năng TTFT và TPOT trên hạ tầng được chỉ định.

Cấu hình Môi trường Hệ thống Linux (Thực thi trước khi khởi chạy Python):

```
Bash
# Triệt tiêu hiện tượng Oversubscription của đa luồng CPU
export OMP_NUM_THREADS=
```

```

export MKL_NUM_THREADS=
export OPENBLAS_NUM_THREADS=
export RAY_DEDUP_LOGS=0

# Ép buộc Kiến trúc Hopper sử dụng thư viện mạnh nhất và Offload bộ lấy mẫu
export VLLM_ATTENTION_BACKEND=FLASHINFER
export VLLM_FLASH_ATTN_VERSION=
export VLLM_USE_FLASHINFER_SAMPLER=

# Đóng băng hệ thống viễn thám nền
export VLLM_NO_USAGE_STATS=
export VLLM_DO_NOT_TRACK=

```

Tham số khởi chạy Khung Ứng Dụng (vLLM Engine Config):

```

Bash
python -m vllm.entrypoints.openai.api_server \
--model <tên-mô-hình-dã-lượng-tự-hoà> \
--quantization fp8 \
--kv-cache-dtype fp8 \
# --- Kiến trúc CPU, Tiến trình và I/O ---
--disable-frontend-multiprocessing \
--distributed-executor-backend mp \
--disable-log-stats \
--disable-log-requests \
# --- Chiến lược VRAM & Đồ thị CUDA (Bóp Sequence, Nơi VRAM) ---
--gpu-memory-utilization 0.96 \
--max-num-seqs 16 \
--enforce-eager False \
# --- Tinh chỉnh Lịch trình & PagedAttention cho CPU Yếu ---
--block-size 32 \
--enable-chunked-prefill \
--max-num-batched-tokens 4096 \
--disable-chunked-mm-input \
--no-enable-prefix-caching \

```

```
# --- Giải mã Đầu cơ Không Mô hình (N-Gram Tần số thấp) ---
```

```
speculative_decode": {"method": "ngram", "num_speculative_tokens": 3, "prompt_lookup_max": 2}
```

Thông qua việc ép buộc API client gửi các truy vấn theo cấu hình Greedy Search (temperature: 0.0) và không lấy lại logprobs, kết hợp với sự điều hướng khắt khe về phân bổ phần cứng, hiện tượng "đói lệnh" của GPU H200 sẽ bị vô hiệu hóa. Hệ thống sẽ khai thác triệt để những byte VRAM cuối cùng và những chu kỳ xung nhịp quý giá từ 3 CPU cores, thiết lập một trạng thái hoạt động hiệu quả tối đa mà không vi phạm tính toàn vẹn chức năng của một nền tảng phục vụ LLM.

Nguồn trích dẫn

1. vLLM × HPC-Ops: High-Performance Attention and MoE Backends from Tencent Hunyuan, <https://vllm.ai/blog/2026-07-06-vllm-hpc-ops>
2. LLM Inference Optimization and Quantization 2026 | Zylus Research, <https://zylus.ai/research/2026-01-15-llm-inference-optimization/>
3. Installation with CPU - vLLM Documentation, https://docs.vllm.ai/en/v0.6.6/getting_started/cpu-installation.html
4. Understanding vLLM Scheduling: Token Budgets, Chunked Prefill, and Policies, <https://audreywongkg.medium.com/understanding-vllm-scheduling-token-budgets-chunked-prefill-and-policies-2c879e3980e3>
5. CPU Binding - vLLM Ascend, https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/cpu_binding.html
6. Inside vLLM: Anatomy of a High-Throughput LLM Inference System, <https://vllm.ai/blog/2025-09-05-anatomy-of-vllm>
7. CPU utilization is extremely high during inference and becomes the primary performance bottleneck - vLLM Forums, <https://discuss.vllm.ai/t/cpu-utilization-is-extremely-high-during-inference-and-becomes-the-primary-performance-bottleneck/1876>
8. Why vLLM Scales: Paging the KV-Cache for Faster LLM Inference - Andrey Krisanov, <https://akrisanov.com/vllm/>
9. Efficient Memory Management for Large Language Model Serving with PagedAttention - 3D Virtual and Augmented Reality, <https://3dvar.com/Kwon2023Efficient.pdf>
10. Optimization and Tuning - vLLM Documentation, <https://docs.vllm.ai/en/v0.8.0/performance/optimization.html>
11. Building Clean, Maintainable vLLM Modifications Using the Plugin System | vLLM Blog, <https://vllm.ai/blog/2025-11-20-vllm-plugin-system>
12. VLLM - 0.3.0 - Hexdocs, <https://hexdocs.pm/vllm/VLLM.epub>
13. OpenAI-Compatible Server - vLLM Documentation, https://docs.vllm.ai/en/v0.8.3/serving/openai_compatible_server.html
14. OpenAI-Compatible Server - vLLM Documentation,

- https://docs.vllm.ai/en/v0.8.5/serving/openai_compatible_server.html
15. Fast vLLM: cutting cold starts by 20x | Joel Smith, https://j9s.io/posts/fast_vllm/
 16. Chapter 3. vLLM server usage - Red Hat Documentation, https://docs.redhat.com/en/documentation/red_hat_ai_inference_server/3.3/html/vllm_server_arguments/vllm-server-usage_server-arguments
 17. vllm.entrypoints.openai.api_server, https://docs.vllm.ai/en/v0.9.0/api/vllm/entrypoints/openai/api_server.html
 18. vllm serve - vLLM Documentation, <https://docs.vllm.ai/en/v0.10.1/cli/serve.html>
 19. Remove Ray for the dependency · Issue #208 · vllm-project/vllm - GitHub, <https://github.com/vllm-project/vllm/issues/208>
 20. [RFC]: Revamp Ray Distributed Executor Backend (from Ray team) · Issue #35848 · vllm-project/vllm - GitHub, <https://github.com/vllm-project/vllm/issues/35848>
 21. Engine Arguments - vLLM Documentation, https://docs.vllm.ai/en/v0.5.5/models/engine_args.html
 22. [RFC]: Model Parallelism with Single Worker using SPMD · Issue #18009 · vllm-project/vllm, <https://github.com/vllm-project/vllm/issues/18009>
 23. Thread Usage Best Practices | Ohio Supercomputer Center, https://www.osc.edu/supercomputing/knowledge-base/thread_usage_best_practices
 24. The current vLLM CPU backend is not working properly - #2 by RunLLM - General, <https://discuss.vllm.ai/t/the-current-vllm-cpu-backend-is-not-working-properly/2708/2>
 25. Common Optimal Environment Variable Settings - 57300, <https://docs.amd.com/r/en-US/57300-ZenDNN-user-guide/Common-Optimal-Environment-Variable-Settings>
 26. Initial release: vLLM container for NVIDIA GB10 Blackwell (DGX Spark) - Hugging Face, <https://huggingface.co/Hellohal2064/vllm-dgx-spark-gb10/commit/ff9ba8fb4d752f7540fbf481dd157f20c864d9fe>
 27. [Usage]: Free GPU memory when using CUDA graphs · Issue #14632 · vllm-project/vllm, <https://github.com/vllm-project/vllm/issues/14632>
 28. GPU Memory Calculation and Configuration - vLLM-Omni, https://docs.vllm.ai/projects/vllm-omni/en/latest/configuration/gpu_memory_utilization/
 29. Automatic GPU inference tuning with vLLM planner in Foundry Local - Microsoft Learn, <https://learn.microsoft.com/en-us/azure/azure-sovereign-clouds/private/foundry-local/concept-gpu-inference-planner>
 30. Speculative Decoding for Local LLMs Was a Mixed Bag. Then DFlash Landed in vLLM, <https://allenkuo.medium.com/when-speculative-decoding-helps-local-llms-and-when-it-doesnt-5c41dd804e4b>
 31. LLM Serving Optimization: Continuous Batching, PagedAttention, and Chunked Prefill on H100 (2026) | Spheron Blog,

- <https://www.spheron.network/blog/llm-serving-optimization-continuous-batching-paged-attention/>
32. FP4 on DGX Spark — Why It Doesn't Scale Like You'd Expect, <https://forums.developer.nvidia.com/t/fp4-on-dgx-spark-why-it-doesnt-scale-like-you-d-expect/360142>
 33. vLLM for beginners: Key Features & Performance Optimization(PartII) - Cloudthrill, <https://cloudthrill.ca/what-is-vllm-features>
 34. The Anatomy of a Triton Attention Kernel - arXiv, <https://arxiv.org/html/2511.11581v1>
 35. Deploy FlashInfer on GPU Cloud: LLM Inference Kernels for vLLM and SGLang (2026 Guide) | Spheron Blog, <https://www.spheron.network/blog/deploy-flashinfer-gpu-cloud-llm-inference-kernels/>
 36. Environment Variables - vLLM Documentation, https://docs.vllm.ai/en/v0.8.3/serving/env_vars.html
 37. Chapter 4. Environment variables | vLLM server arguments | Red Hat AI Inference Server, https://docs.redhat.com/en/documentation/red_hat_ai_inference_server/3.0/html/vllm_server_arguments/environment_variables-server-arguments
 38. Flash Attention 3 (FA3) Support #12429 - vllm-project/vllm - GitHub, <https://github.com/vllm-project/vllm/issues/12429>
 39. [Bug]: FlashInfer sampler JIT fails on Blackwell SM120f with CUDA 13.3: CUDA compiler and toolkit headers are incompatible · Issue #44305 · vllm-project/vllm - GitHub, <https://github.com/vllm-project/vllm/issues/44305>
 40. vLLM Explained: PagedAttention and Continuous Batching - Runpod, <https://www.runpod.io/articles/guides/vllm-pagedattention-continuous-batching>
 41. Engine Arguments - vLLM Documentation, https://docs.vllm.ai/en/v0.6.1/models/engine_args.html
 42. Why is chunked prefill always enabled in V1? - General - vLLM Forums, <https://discuss.vllm.ai/t/why-is-chunked-prefill-always-enabled-in-v1/1628>
 43. vllm/docs/configuration/optimization.md at main - GitHub, <https://github.com/vllm-project/vllm/blob/main/docs/configuration/optimization.md>
 44. vLLM Optimization Techniques: 5 Practical Methods to Improve Performance - Jarvis Labs, <https://jarvislabs.ai/blog/vllm-optimization-techniques>
 45. [Performance]: High VRAM overhead in Diffusion Stage and ineffectiveness of `gpu\_memory\_utilization` · Issue #1818 · vllm-project/vllm-omni - GitHub, <https://github.com/vllm-project/vllm-omni/issues/1818>
 46. Distributed vLLM Inference - Bristol Centre for Supercomputing Documentation, <https://docs.isambard.ac.uk/user-documentation/tutorials/distributed-inference/>
 47. Speculative Decoding in vLLM: Complete Guide to Faster LLM Inference | Jarvis Labs Blog, <https://jarvislabs.ai/blog/speculative-decoding-vllm-faster-llm-inference>
 48. Speculative Decoding: Achieving 2-3x LLM Inference Speedup | Introl Blog, <https://introl.com/blog/speculative-decoding-llm-inference-speedup-guide-2025>

49. N-Gram Speculation - vLLM Documentation,
https://docs.vllm.ai/en/latest/features/speculative_decoding/n_gram/
50. Draft Models - vLLM Documentation,
https://docs.vllm.ai/en/latest/features/speculative_decoding/draft_model/
51. Speculative decoding - vLLM Documentation,
https://docs.vllm.ai/en/v0.6.6/usage/spec_decode.html
52. Fastest Speculative Decoding in vLLM with Arctic Inference and Arctic Training - Snowflake,
<https://www.snowflake.com/en/blog/engineering/fast-speculative-decoding-vllm-arctic/>
53. vllm bench throughput, <https://docs.vllm.ai/en/v0.14.0/cli/bench/throughput/>
54. 5 steps to triage vLLM performance - Red Hat Developer,
<https://developers.redhat.com/articles/2026/03/09/5-steps-triage-vllm-performance>
55. Code Review: Deep Dive into vLLM's Architecture and Implementation Analysis of OpenAI-Compatible Serving (1/2) | Zerohertz,
<https://zerohertz.github.io/vllm-openai-1/>
56. Environment Variables - vLLM Documentation,
https://docs.vllm.ai/en/v0.6.3.post1/serving/env_vars.html